In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
```

In [3]:
```
df = pd.read_csv(r"D:\Churn_Modelling.csv")

print("Dataset Head:")
print(df.head())
print("\nDataset Info:")
print(df.info())
```

```
Dataset Head:
   RowNumber  CustomerId   Surname  CreditScore Geography  Gender  Age  \
0          1    15634602  Hargrave          619    France  Female   42
1          2    15647311      Hill          608     Spain  Female   41
2          3    15619304      Onio          502    France  Female   42
3          4    15701354      Boni          699    France  Female   39
4          5    15737888  Mitchell          850     Spain  Female   43

   Tenure    Balance  NumOfProducts  HasCrCard  IsActiveMember  \
0       2       0.00              1          1               1
1       1   83807.86              1          0               1
2       8  159660.80              3          1               0
3       1       0.00              2          0               0
4       2  125510.82              1          1               1

   EstimatedSalary  Exited
0        101348.88       1
1        112542.58       0
2        113931.57       1
3         93826.63       0
4         79084.10       0

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   RowNumber        10000 non-null  int64
 1   CustomerId       10000 non-null  int64
 2   Surname          10000 non-null  object
 3   CreditScore      10000 non-null  int64
 4   Geography        10000 non-null  object
 5   Gender           10000 non-null  object
 6   Age              10000 non-null  int64
 7   Tenure           10000 non-null  int64
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64
 10  HasCrCard        10000 non-null  int64
 11  IsActiveMember   10000 non-null  int64
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
None
```

In [4]:
```
df = df.drop(columns=["RowNumber", "CustomerId", "Surname"])
```

In [5]:
```
df = pd.get_dummies(df, columns=["Geography", "Gender"], drop_first=True)
```

In [6]:
```
X = df.drop(columns=["Exited"])
y = df["Exited"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
```

```
)

print(f"Training set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")
```

```
Training set shape: (8000, 11)
Testing set shape: (2000, 11)
```

In [10]:
```
improved_model = Sequential([
    Dense(32, activation="relu", input_shape=(X_train_scaled.shape[1],)),
    Dropout(0.3),
    Dense(16, activation="relu"),
    Dropout(0.2),
    Dense(8, activation="relu"),
    Dense(1, activation="sigmoid")
])

improved_model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accu

early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=10,
    restore_best_weights=True
)

history = improved_model.fit(
    X_train_scaled, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stopping],
    verbose=1
)
```

```
Epoch 1/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.7883 - loss: 0.5425 - val_acc
uracy: 0.8031 - val_loss: 0.4544
Epoch 2/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8041 - loss: 0.4583 - val_acc
uracy: 0.8331 - val_loss: 0.4157
Epoch 3/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.8077 - loss: 0.4379 - val_acc
uracy: 0.8369 - val_loss: 0.4091
Epoch 4/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8117 - loss: 0.4282 - val_acc
uracy: 0.8394 - val_loss: 0.3976
Epoch 5/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8159 - loss: 0.4200 - val_acc
uracy: 0.8494 - val_loss: 0.3883
Epoch 6/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8206 - loss: 0.4097 - val_acc
uracy: 0.8569 - val_loss: 0.3781
Epoch 7/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.8244 - loss: 0.4028 - val_acc
uracy: 0.8606 - val_loss: 0.3691
Epoch 8/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8295 - loss: 0.3958 - val_acc
uracy: 0.8612 - val_loss: 0.3612
Epoch 9/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8316 - loss: 0.3867 - val_acc
uracy: 0.8612 - val_loss: 0.3540
Epoch 10/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8331 - loss: 0.3839 - val_acc
uracy: 0.8669 - val_loss: 0.3500
Epoch 11/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8353 - loss: 0.3785 - val_acc
uracy: 0.8700 - val_loss: 0.3440
Epoch 12/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8405 - loss: 0.3771 - val_acc
uracy: 0.8662 - val_loss: 0.3406
Epoch 13/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8381 - loss: 0.3738 - val_acc
uracy: 0.8669 - val_loss: 0.3383
Epoch 14/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8380 - loss: 0.3768 - val_acc
uracy: 0.8650 - val_loss: 0.3391
Epoch 15/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8427 - loss: 0.3735 - val_acc
uracy: 0.8687 - val_loss: 0.3362
Epoch 16/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8438 - loss: 0.3689 - val_acc
uracy: 0.8687 - val_loss: 0.3337
Epoch 17/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8439 - loss: 0.3694 - val_acc
uracy: 0.8675 - val_loss: 0.3325
Epoch 18/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8462 - loss: 0.3664 - val_acc
uracy: 0.8656 - val_loss: 0.3344
Epoch 19/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8459 - loss: 0.3648 - val_acc
```

```
uracy: 0.8687 - val_loss: 0.3324
Epoch 20/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8487 - loss: 0.3669 - val_acc
uracy: 0.8712 - val_loss: 0.3315
Epoch 21/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8461 - loss: 0.3671 - val_acc
uracy: 0.8656 - val_loss: 0.3320
Epoch 22/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8481 - loss: 0.3639 - val_acc
uracy: 0.8650 - val_loss: 0.3349
Epoch 23/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8516 - loss: 0.3601 - val_acc
uracy: 0.8681 - val_loss: 0.3312
Epoch 24/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8458 - loss: 0.3643 - val_acc
uracy: 0.8675 - val_loss: 0.3314
Epoch 25/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8555 - loss: 0.3561 - val_acc
uracy: 0.8694 - val_loss: 0.3299
Epoch 26/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8512 - loss: 0.3618 - val_acc
uracy: 0.8669 - val_loss: 0.3306
Epoch 27/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8525 - loss: 0.3598 - val_acc
uracy: 0.8700 - val_loss: 0.3294
Epoch 28/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8486 - loss: 0.3612 - val_acc
uracy: 0.8669 - val_loss: 0.3343
Epoch 29/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.8480 - loss: 0.3621 - val_acc
uracy: 0.8656 - val_loss: 0.3313
Epoch 30/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8511 - loss: 0.3561 - val_acc
uracy: 0.8650 - val_loss: 0.3291
Epoch 31/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.8522 - loss: 0.3567 - val_acc
uracy: 0.8694 - val_loss: 0.3295
Epoch 32/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8520 - loss: 0.3510 - val_acc
uracy: 0.8700 - val_loss: 0.3284
Epoch 33/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8544 - loss: 0.3521 - val_acc
uracy: 0.8712 - val_loss: 0.3275
Epoch 34/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8491 - loss: 0.3559 - val_acc
uracy: 0.8662 - val_loss: 0.3302
Epoch 35/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.8533 - loss: 0.3550 - val_acc
uracy: 0.8675 - val_loss: 0.3284
Epoch 36/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8530 - loss: 0.3555 - val_acc
uracy: 0.8662 - val_loss: 0.3297
Epoch 37/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.8558 - loss: 0.3527 - val_acc
uracy: 0.8687 - val_loss: 0.3273
Epoch 38/100
```

```
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8523 - loss: 0.3544 - val_acc
uracy: 0.8687 - val_loss: 0.3266
Epoch 39/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8523 - loss: 0.3555 - val_acc
uracy: 0.8700 - val_loss: 0.3269
Epoch 40/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.8544 - loss: 0.3505 - val_acc
uracy: 0.8694 - val_loss: 0.3242
Epoch 41/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8494 - loss: 0.3517 - val_acc
uracy: 0.8694 - val_loss: 0.3277
Epoch 42/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8542 - loss: 0.3480 - val_acc
uracy: 0.8712 - val_loss: 0.3257
Epoch 43/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.8509 - loss: 0.3542 - val_acc
uracy: 0.8731 - val_loss: 0.3276
Epoch 44/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8537 - loss: 0.3550 - val_acc
uracy: 0.8700 - val_loss: 0.3248
Epoch 45/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8556 - loss: 0.3514 - val_acc
uracy: 0.8700 - val_loss: 0.3276
Epoch 46/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8572 - loss: 0.3496 - val_acc
uracy: 0.8712 - val_loss: 0.3262
Epoch 47/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.8530 - loss: 0.3523 - val_acc
uracy: 0.8681 - val_loss: 0.3255
Epoch 48/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8564 - loss: 0.3492 - val_acc
uracy: 0.8675 - val_loss: 0.3259
Epoch 49/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8552 - loss: 0.3504 - val_acc
uracy: 0.8662 - val_loss: 0.3250
Epoch 50/100
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8556 - loss: 0.3492 - val_acc
uracy: 0.8669 - val_loss: 0.3258
```

In [7]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [11]:
```
y_pred_probs = improved_model.predict(X_test_scaled)
y_pred = (y_pred_probs > 0.5).astype(int)

acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
cr = classification_report(y_test, y_pred)
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
```

In [12]:
```
print("\n==================================")
print("FINAL MODEL RESULTS")
print("==================================")
print(f"Accuracy Score: {acc:.4f} ({acc * 100:.2f}%)")
```

```
print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(cr)

# Plotting Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Stayed", "Exited"]
plt.title("Confusion Matrix - ANN Churn Classifier")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

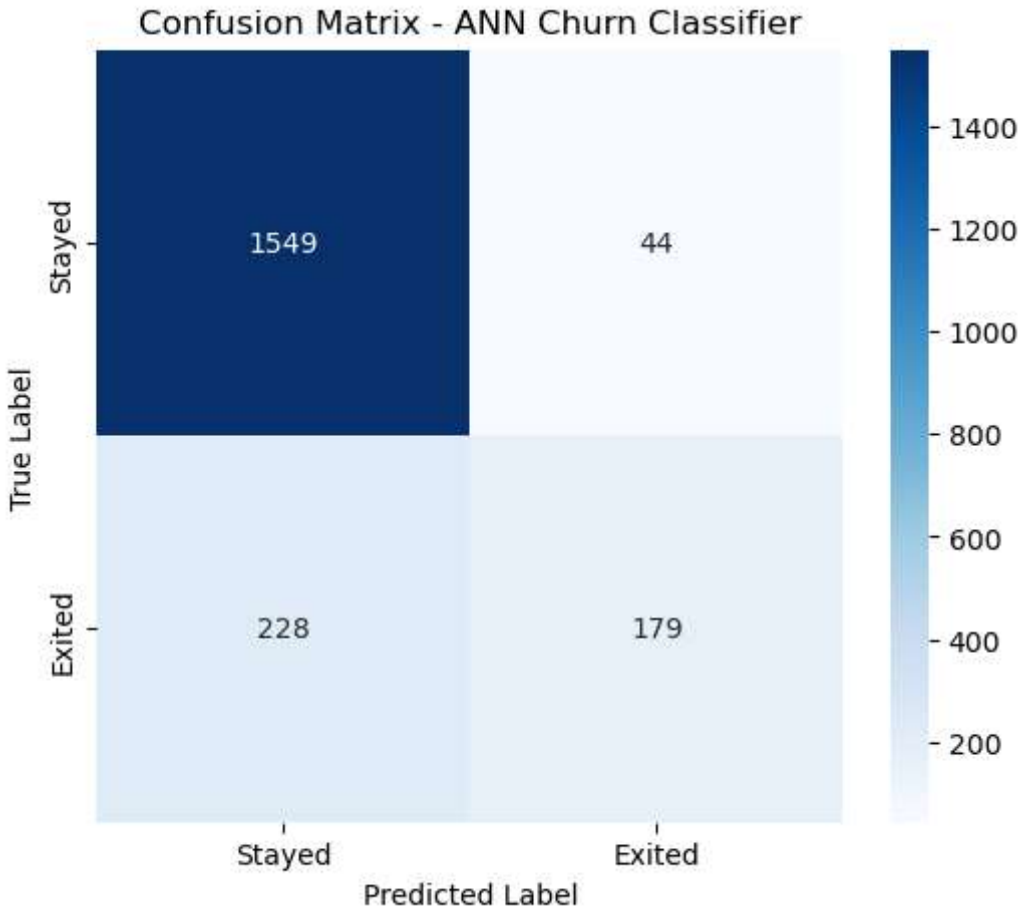
```
==================================
FINAL MODEL RESULTS
==================================
Accuracy Score: 0.8640 (86.40%)

Confusion Matrix:
[[1549   44]
 [ 228  179]]

Classification Report:
              precision    recall  f1-score   support

           0       0.87      0.97      0.92      1593
           1       0.80      0.44      0.57       407

    accuracy                           0.86      2000
   macro avg       0.84      0.71      0.74      2000
weighted avg       0.86      0.86      0.85      2000
```


Confusion Matrix - ANN Churn Classifier
1549
44
228
179
Stayed
Exited
True Label
Stayed
Exited
Predicted Label
1400
1200
1000
800
600
400
200

In [ ]: